IT Workshop II (Cloud Computing): Python for Cloud

September 3, 2026

Amazon AutoScaling

- **AutoScaling Group:**
  - An Auto Scaling Group in AWS is a collection of Amazon EC2 instances that are managed as a group for the purposes of automatic scaling and management.
  - It is a key feature of the Amazon EC2 Auto Scaling service, which ensures that you have the correct number of EC2 instances available to handle the load for your application.
- **Automatic Scaling:**
  - **Scale Out (Increase Capacity):** The group can automatically launch new EC2 instances when demand increases.
  - **Scale In (Decrease Capacity):** It can terminate instances when demand decreases to save costs.

Amazon AutoScaling

- **Desired Capacity:** The number of instances that you want running at all times.
- **Minimum Capacity:** The minimum number of instances that should always be running.
- **Maximum Capacity:** The maximum number of instances that can be running in the group.
- **Launch Configuration or Launch Template:** When creating an Auto Scaling Group, you define how new instances should be launched using a launch configuration or launch template. This includes the AMI, instance type, security groups, key pairs, and other configuration details.

Amazon AutoScaling

- **Load Balancing:** An Auto Scaling Group can be associated with an Elastic Load Balancer (ELB), which automatically distributes incoming traffic across the instances in the group.
- **Scaling Policies:** You can define scaling policies that determine when and how the Auto Scaling Group should scale in or out. These policies can be triggered by CloudWatch alarms based on metrics like CPU utilization, request count, or custom metrics.
- **Availability Zones:** Auto Scaling Groups can span multiple Availability Zones within a region to improve fault tolerance and availability.

Amazon AutoScaling

- **Example Use Cases:**
  - **Handling Variable Traffic:** Automatically scale the number of instances up during peak traffic periods (e.g., during a sale or a launch) and scale down during off-peak times to reduce costs.
  - **Ensuring Availability:** Maintain application availability by automatically replacing failed instances with healthy ones.
  - **Disaster Recovery:** Automatically distribute instances across multiple Availability Zones to protect against zone-specific failures.
- **Benefits:**
  - Only run the number of instances you need, scaling in when demand is low.
  - Automatically adjust capacity to meet traffic demands, ensuring consistent application performance.
  - Distribute instances across multiple Availability Zones and automatically replace unhealthy instances.

CloudWatch Alarm

- A CloudWatch Alarm in AWS is a monitoring tool that watches a specific metric (such as CPU utilization, network traffic, or custom metrics) and performs one or more actions when the metric reaches a predefined threshold.
- CloudWatch Alarms are part of Amazon CloudWatch, which is a service for monitoring and managing various AWS resources and applications.
- **Features:**
  - A CloudWatch Alarm monitors a metric over a period of time, checking if it meets or exceeds a certain threshold. Metrics can be system-level (e.g., EC2 instance CPU usage) or application-specific (e.g., custom metrics you define).
  - The alarm is configured with a threshold that determines when it should transition between states. For example, you might set a threshold for CPU utilization to 80

CloudWatch Alarm

- **Features:**
  - The alarm evaluates the metric over a series of time periods. For example, you might require that the metric exceed the threshold for three consecutive five-minute periods before triggering the alarm.
  - **Alarm States:**
    - **OK:** The metric is within the desired range.
    - **ALARM:** The metric has crossed the threshold.
    - **INSUFFICIENT_DATA:** Not enough data is available to evaluate the metric.
  - **Actions:** When an alarm state changes (e.g., from OK to ALARM), it can trigger actions such as:
    - **Sending Notifications:** Via Amazon Simple Notification Service (SNS) to alert you or your team.
    - **Auto Scaling:** Adjust the number of instances in an Auto Scaling group.
    - **EC2 Actions:** Stop, terminate, reboot, or recover an EC2 instance.

CloudWatch Alarm

- **Actions:** When an alarm state changes (e.g., from OK to ALARM), it can trigger actions such as:
  - **Custom Actions:** Trigger a Lambda function or other automated processes.
- **Example Use Cases:**
  - **Auto Scaling:** Automatically increase or decrease the number of running EC2 instances in response to workload changes.
  - **Alerting:** Get notified immediately if a critical application or service metric, such as latency or error rate, exceeds acceptable limits.
  - **Resource Management:** Automatically shut down underutilized resources to save costs or reboot instances that are unresponsive.

Amazon RDS

- Amazon RDS is a web service that allows you to create instances of MySQL, Oracle, Microsoft SQL Server, PostgreSQL, MariaDB etc. in the cloud.
- With RDS, developers can easily set up, operate, and scale a relational database in the cloud.
- A connection to RDS service is established by calling boto.rds.connect_to_region function.
- The RDS region, AWS access key and AWS secret key are passed to this function.
- After connecting to RDS service, the conn.create_dbinstance function is called to launch a new RDS instance.

The End